Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-216БВ-24

Студент: ГеннихА.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата:25.11.25

Москва, 2025

Постановка задачи

Вариант 31.

Расчет значения числа e (основание натурального логарифма):

Сигнатура функции: float e(int x);

Реализация №1: (1 + 1/x)^x

Реализация №2: Сумма ряда по n от 0 до x, где элементы ряда равны: (1 / (n!))

Подсчет площади плоской геометрической фигуры по двум сторонам:

Сигнатура функции: float area(float a, float b);

Реализация №1: Фигура прямоугольник

Реализация №2: Фигура прямоугольный треугольник

Общий метод и алгоритм решения

Использованные системные вызовы:

dlopen() - Загружает динамическую библиотеку (.so файл) в память процесса. Возвращает указатель (handle) на загруженную библиотеку;

dlsym() - Получает адрес функции или переменной из загруженной библиотеки по её имени. Возвращает указатель на функцию;

dlclose() - Выгружает динамическую библиотеку из памяти, освобождая занятые ресурсы;

dlerror() - Возвращает строку с описанием последней ошибки при работе с динамическими библиотеками

mmap() - Отображает файл библиотеки в память процесса (вызывается неявно внутри dlopen);

munmap() - Удаляет отображение файла из памяти (вызывается неявно внутри dlclose)

e_impl1() - функция, первая реализация вычисления числа e по формуле (1 + 1/x)^x

e_impl2() - функция, вторая реализация вычисления числа e через сумму ряда 1/n! от 0 до x;

area_impl1() - функция, первая реализация вычисления площади прямоугольника (a * b);

area_impl2() - функция, вторая реализация вычисления площади прямоугольного треугольника ((a * b) / 2);

load_libraries() - функция для загрузки библиотек с использованием dlopen и dlsym;

switch_implementations() - функция для переключения между реализациями (команда 0);

Общий метод решения заключается в создании двух динамических библиотек с разными реализациями функций для вычисления числа e (первая реализация: (1 + 1/x)^x, вторая реализация: сумма ряда 1/n! от 0 до x) и вычисления площади геометрической фигуры (первая реализация: прямоугольник a*b, вторая реализация: прямоугольный треугольник a*b/2), после чего разрабатываются две тестовые программы: программа №1 использует эти библиотеки на этапе компиляции (статическая линковка), где код библиотек встраивается в исполняемый файл и нельзя сменить реализацию без перекомпиляции, а программа №2 загружает библиотеки во время выполнения с использованием системных вызовов ОС (dlopen, dlsym, dlclose), что позволяет динамически переключаться между реализациями командой "0" без перезапуска программы. Обе программы обрабатывают пользовательские команды: "1 x" для вычисления e и "2 a b" для вычисления площади, выводя результаты на экран. В результате проводится анализ двух подходов

Код программы

Area_calculator.h

```
#ifndef AREA_CALCULATOR_H
#define AREA_CALCULATOR_H

float area_impl1(float a, float b);
float area_impl2(float a, float b);

#endif
```

E_calculator.h

```
#ifndef E_CALCULATOR_H
#define E_CALCULATOR_H


float e_impl1(int x);
float e_impl2(int x);


#endif
```

Area.impl1.c

```
#include "area_calculator.h"

float area_impl1(float a, float b) {
    if (a <= 0 || b <= 0) return 0.0f;
    return a * b;
}
```

Area.impl2.c

```
#include "area_calculator.h"

float area_impl2(float a, float b) {
    if (a <= 0 || b <= 0) return 0.0f;
    return 0.5f * a * b;
}
```

E_impl1.c

```
#include <math.h>
#include "e_calculator.h"

float e_impl1(int x) {
```

```
    if (x <= 0) return 0.0f;
    return powf(1.0f + 1.0f/x, x);
}
```

E_imple2.c

```
#include "e_calculator.h"

float e_impl2(int x) {
    if (x < 0) return 0.0f;

    float result = 0.0f;
    float factorial = 1.0f;

    for (int n = 0; n <= x; n++) {
        if (n > 0) {
            factorial *= n;
        }
        result += 1.0f / factorial;
    }

    return result;
}
```

Prog1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "e_calculator.h"
#include "area_calculator.h"

int main() {
    char command[256];
    char *token;
    int choice;

    printf("========================================\n");
    printf("Программа №1 (статическая линковка)\n");
    printf("========================================\n");
    printf("Команды:\n");
    printf("  1 x - вычислить e (реализация 1)\n");
    printf("  2 a b - вычислить площадь прямоугольника\n");
    printf("  q - выход\n\n");
```

```
while (1) {
    printf("prog1> ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = 0;

    if (strcmp(command, "q") == 0 || strcmp(command, "Q") == 0) {
        break;
    }

    token = strtok(command, " ");
    if (token == NULL) continue;

    choice = atoi(token);

    switch (choice) {
        case 1: {
            token = strtok(NULL, " ");
            if (token == NULL) {
                printf("Ошибка: не указан аргумент x\n");
                break;
            }
            int x = atoi(token);
            float result = e_impl1(x);
            printf("e = %.6f (приближение 1: (1+1/%d)^%d)\n", result, x, x);
            break;
        }

        case 2: {
            token = strtok(NULL, " ");
            if (token == NULL) {
                printf("Ошибка: не указан аргумент a\n");
                break;
            }
            float a = atof(token);

            token = strtok(NULL, " ");
            if (token == NULL) {
                printf("Ошибка: не указан аргумент b\n");
                break;
            }
            float b = atof(token);

            float result = area_impl1(a, b);
```

```
            printf("Площадь прямоугольника = %.2f\n", result);
            break;
        }

        default:
            printf("Неизвестная команда\n");
            break;
    }
}

printf("Программа завершена\n");
return 0;
}
```

Prog2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dlfcn.h>

typedef float (*e_func)(int);
typedef float (*area_func)(float, float);

e_func current_e = NULL;
area_func current_area = NULL;

void *e_lib_handle = NULL;
void *area_lib_handle = NULL;

int load_libraries(const char *e_lib_path, const char *e_func_name,
                   const char *area_lib_path, const char *area_func_name) {

    if (e_lib_handle != NULL) {
        dlclose(e_lib_handle);
    }

    e_lib_handle = dlopen(e_lib_path, RTLD_LAZY);
    if (!e_lib_handle) {
        fprintf(stderr, "Ошибка загрузки %s: %s\n", e_lib_path, dlerror());
        return -1;
    }

    current_e = (e_func)dlsym(e_lib_handle, e_func_name);
```

```
    if (!current_e) {
        fprintf(stderr, "Ошибка получения функции %s: %s\n", e_func_name,
dlerror());
        return -1;
    }

    if (area_lib_handle != NULL) {
        dlclose(area_lib_handle);
    }

    area_lib_handle = dlopen(area_lib_path, RTLD_LAZY);
    if (!area_lib_handle) {
        fprintf(stderr, "Ошибка загрузки %s: %s\n", area_lib_path, dlerror());
        return -1;
    }

    current_area = (area_func)dlsym(area_lib_handle, area_func_name);
    if (!current_area) {
        fprintf(stderr, "Ошибка получения функции %s: %s\n", area_func_name,
dlerror());
        return -1;
    }

    return 0;
}

void switch_implementations() {
    static int impl_num = 1;

    impl_num = (impl_num == 1) ? 2 : 1;

    char e_func_name[20];
    sprintf(e_func_name, "e_impl%d", impl_num);

    char area_func_name[20];
    sprintf(area_func_name, "area_impl%d", impl_num);

    char e_lib_path[50];
    sprintf(e_lib_path, "./libe_impl%d.so", impl_num);

    char area_lib_path[50];
    sprintf(area_lib_path, "./libarea_impl%d.so", impl_num);
```

```
    if (load_libraries(e_lib_path, e_func_name, area_lib_path, area_func_name) ==
0) {
        printf(">>> Переключено на реализацию %d\n", impl_num);
        printf("    e: %s\n", (impl_num == 1) ? "(1+1/x)^x" : "сумма ряда 1/n!");
        printf("    площадь: %s\n", (impl_num == 1) ? "прямоугольник" :
"прямоугольный треугольник");
    }
}

int main() {
    char command[256];
    char *token;
    int choice;

    if (load_libraries("./libe_impl1.so", "e_impl1",
                    "./libarea_impl1.so", "area_impl1") != 0) {
        fprintf(stderr, "Не удалось загрузить библиотеки. Сначала выполните
'make'\n");
        return 1;
    }

    printf("========================================\n");
    printf("Программа №2 (динамическая загрузка)\n");
    printf("========================================\n");
    printf("Команды:\n");
    printf("  0 - переключить реализации\n");
    printf("  1 x - вычислить e\n");
    printf("  2 a b - вычислить площадь\n");
    printf("  q - выход\n\n");
    printf("Текущая реализация: 1\n");
    printf("  e: (1+1/x)^x\n");
    printf("  площадь: прямоугольник\n\n");

    while (1) {
        printf("prog2> ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = 0;

        if (strcmp(command, "q") == 0 || strcmp(command, "Q") == 0) {
            break;
        }

        token = strtok(command, " ");
```

```
if (token == NULL) continue;

choice = atoi(token);

switch (choice) {
    case 0:
        switch_implementations();
        break;

    case 1: {
        token = strtok(NULL, " ");
        if (token == NULL) {
            printf("Ошибка: не указан аргумент x\n");
            break;
        }
        int x = atoi(token);
        if (current_e) {
            float result = current_e(x);
            printf("e = %.6f\n", result);
        }
        break;
    }

    case 2: {
        token = strtok(NULL, " ");
        if (token == NULL) {
            printf("Ошибка: не указан аргумент a\n");
            break;
        }
        float a = atof(token);

        token = strtok(NULL, " ");
        if (token == NULL) {
            printf("Ошибка: не указан аргумент b\n");
            break;
        }
        float b = atof(token);

        if (current_area) {
            float result = current_area(a, b);
            printf("Площадь = %.2f\n", result);
        }
        break;
```

```
        }

        default:
            printf("Неизвестная команда\n");
            break;
    }
  }

  if (e_lib_handle) dlclose(e_lib_handle);
  if (area_lib_handle) dlclose(area_lib_handle);

  printf("Программа завершена\n");
  return 0;
}
```

Протокол работы программы

Тестирование:

```
hoodrich@DESKTOP-DUQ2OCS:~/OSI_LAB/laba_4/build$ make run_prog1
[ 20%] Built target area_impl1_obj
[ 40%] Built target e_impl1_obj
[ 80%] Built target prog1
[100%] Запуск программы 1 (статическая линковка)
========================================
Программа №1 (статическая линковка)
========================================
Команды:
  1 x - вычислить e (реализация 1)
  2 a b - вычислить площадь прямоугольника
  q - выход

prog1> 1 5
e = 2.488321 (приближение 1: (1+1/5)^5)
prog1> й
Неизвестная команда
prog1> q
Программа завершена
[100%] Built target run_prog1
```

Strace:

```
execve("./prog1", ["./prog1"], 0x7ffffc745dd0 /* 37 vars */) = 0
brk(NULL)                               = 0x555fa69b3000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f9c74055000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20203, ...}) = 0
mmap(NULL, 20203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f9c74050000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libm.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=952616, ...}) = 0
mmap(NULL, 950296, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f9c73f67000
mmap(0x7f9c73f77000, 520192, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x10000) = 0x7f9c73f77000
mmap(0x7f9c73ff6000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x8f000) = 0x7f9c73ff6000
mmap(0x7f9c7404e000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0xe7000) = 0x7f9c7404e000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f9c73d55000
mmap(0x7f9c73d7d000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f9c73d7d000
mmap(0x7f9c73f05000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b0000) = 0x7f9c73f05000
mmap(0x7f9c73f54000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7f9c73f54000
mmap(0x7f9c73f5a000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f9c73f5a000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f9c73d52000
arch_prctl(ARCH_SET_FS, 0x7f9c73d52740) = 0
set_tid_address(0x7f9c73d52a10)         = 39650
set_robust_list(0x7f9c73d52a20, 24)     = 0
rseq(0x7f9c73d53060, 0x20, 0, 0x53053053) = 0
mprotect(0x7f9c73f54000, 16384, PROT_READ) = 0
mprotect(0x7f9c7404e000, 4096, PROT_READ) = 0
mprotect(0x555f9948f000, 4096, PROT_READ) = 0
mprotect(0x7f9c7408d000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f9c74050000, 20203)           = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x4), ...}) = 0
getrandom("\x0a\xb8\x2c\x4e\x6d\x0c\x27\xa5", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x555fa69b3000
brk(0x555fa69d4000)                     = 0x555fa69d4000
write(1, "================================"..., 41) = 41
write(1, "\320\237\321\200\320\276\320\263\321\200\320\260\320\274\320\274\320\260 \342\204\2261
(\321\201\321\202\320\260\321"..., 66) = 66
write(1, "================================"..., 41) = 41
write(1, "\320\232\320\276\320\274\320\260\320\275\320\264\321\213:\n", 16) = 16
```

```
write(1, "  1 x - \320\262\321\213\321\207\320\270\321\201\320\273\320\270\321\202\321\214 e (\321\200"..., 54) = 54
write(1, "  2 a b - \320\262\321\213\321\207\320\270\321\201\320\273\320\270\321\202\321\214 \320\277\320"..., 73) = 73
write(1, "  q - \320\262\321\213\321\205\320\276\320\264\n", 17) = 17
write(1, "\n", 1)                    = 1
fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x4), ...}) = 0
write(1, "prog1> ", 7)               = 7
read(0, "1 10\n", 1024)               = 5
write(1, "e = 2.593743 (\320\277\321\200\320\270\320\261\320\273\320\270\320\266\320\265\320\275"..., 53) = 53
write(1, "prog1> ", 7)               = 7
read(0, "2 5 3\n", 1024)              = 6
write(1, "\320\237\320\273\320\276\321\211\320\260\320\264\321\214 \320\277\321\200\321\217\320\274\320\276\321\203\320\263\320\276\320"..., 52) = 52
write(1, "prog1> ", 7)               = 7
read(0, "\320\271\n", 1024)           = 3
write(1, "\320\235\320\265\320\270\320\267\320\262\320\265\321\201\321\202\320\275\320\260\321\217 \320\272\320\276\320\274\320\260\320"..., 38) = 38
write(1, "prog1> ", 7)               = 7
read(0, "q\n", 1024)                  = 2
write(1, "\320\237\321\200\320\276\320\263\321\200\320\260\320\274\320\274\320\260 \320\267\320\260\320\262\320\265\321\200\321\210\320"..., 38) = 38
exit_group(0)                         = ?
+++ exited with 0 +++
```

Вывод

В ходе выполнения лабораторной работы были созданы динамические библиотеки, реализующие два варианта функций: вычисление числа e (первая реализация: (1+1/x)^x, вторая реализация: сумма ряда 1/n!) и вычисление площади геометрической фигуры (первая реализация: прямоугольник, вторая реализация: прямоугольный треугольник). Также были разработаны две тестовые программы: программа №1, использующая библиотеки на этапе компиляции (статическая линковка), и программа №2, загружающая библиотеки во время выполнения с использованием системных вызовов ОС (dlopen, dlsym, dlclose).
В программе №2 реализована возможность переключения между реализациями командой "0" без перезапуска программы, что демонстрирует гибкость динамической загрузки. Проведен анализ двух подходов: статическая линковка обеспечивает простоту и независимость от наличия библиотечных файлов во время выполнения, но не позволяет изменять реализацию без перекомпиляции; динамическая загрузка требует наличия библиотек и использования системных вызовов, но предоставляет гибкость и возможность обновления функционала на лету. С помощью strace были проанализированы системные вызовы, используемые при динамической загрузке библиотек (open, mmap, close), что позволило наглядно увидеть взаимодействие программы с операционной системой.